Open in app

Kawshik Kumar Paul

3 Followers About

Git and GitHub Handbook

Kawshik Kumar Paul Dec 27, 2020 · 12 min read

> *This is a short handbook for git and github. It covers the basic things which are needed to know while working with git and github.*

Git is an open-source, version control tool. **GitHub** is a code hosting platform for version control and collaboration.

Install Git

install git scm from https://git-scm.com/

I am using `git version 2.29.2.windows.1` here.

Open **Git Bash** from **Start Menu**.

To check your version after installation:

```
git --version
```

Set your **username**:

```
git config --global user.name "kawshikbuet17"
```

Set your **email**:

```
git config --global user.email "kawshik.kumar.paul@gmail.com"
```

Now to see git configuration details

```
git config --list
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager-core
credential.https://dev.azure.com.usehttppath=true
filter.lfs.required=true
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
user.name=kawshikbuet17
user.email=kawshik.kumar.paul@gmail.com

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials
$ git config user.name
kawshikbuet17

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials
$ git config user.email
kawshik.kumar.paul@gmail.com
```

Initialize Git

Now go to the specific directory which you want to track with git. Then open **git bash** there and run this command.

```
git status
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials
$ git status
fatal: not a git repository (or any of the parent directories): .git
```

Git will not make any directory automatically tracked. You must tell git to track your directory by a simple command:

```
git init
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials
$ git init
Initialized empty Git repository in C:/Users/ASUS/Desktop/Git tutorials/.git/
```

git init is an initial command of a repository to track. After running this command, your repository is a git repository. Now you start work.

Lets add some files and check `git status` .

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        db.accdb
        first.txt
        myexcel.xlsx

nothing added to commit but untracked files present (use "git add" to track)
```

So, our journey of git has been started. `git status` command is a command which you must know even if your forget your own name.

If you have come to this point, then your are successful. The rests are not so hard.

Commit Changes (That means save-points)

After going to a directory anytime, the first command you must run is `git status` to see the condition of your directory.

Here note that `working tree is clean` means all changes is tracked successfully. `untracked file` means we didn't add it for tracking. `modified` means we added for tracking but didn't commit the changes. `stage` means to add for tracking, `unstage` means not to add for tracking.

Anyway, let's get started with a directory.

```
git status
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   this.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html
```

To stage changes for selective file

```
git add fileName
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   this.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git add .
```

To commit the changes of staged file (**unstaged files** will not participate here):

```
git commit -m "your commit message here"
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
```

```
$ git add .

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git commit -m "commited this.txt index.html"
[master 969f4cd] commited this.txt index.html
 2 files changed, 12 insertions(+)
 create mode 100644 index.html

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Working tree clean means, all changes commited

To stage all changes (not specific file) and commit

```
git add .

git commit -m "type commit message here"
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   this.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git add .

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git commit -m "commited this.txt index.html"
[master 969f4cd] commited this.txt index.html
 2 files changed, 12 insertions(+)
 create mode 100644 index.html

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
nothing to commit, working tree clean
```

After commit, working tree will be clean, that means commit successful.

This can be done faster with one command (skipping `git add .` command)

```
git commit -a -m "type here commit message"
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue1)
$ git status
On branch issue1
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue1)
$ git commit -a -m "this is c5"
[issue1 2331faa] this is c5
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Suppose you want to discard the changes you have made (and didn't staged it I mean didn't add it) and want to get back like last commit (I mean last save-point) :

```
git restore filename
or,
git checkout filename (don't use this)
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   this.txt

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git checkout this.txt
Updated 1 path from the index

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
nothing to commit, working tree clean
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
```

```
  (use "git restore <file>..." to discard changes in working directory)
        modified:   this.txt

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git checkout this.txt
Updated 1 path from the index

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
nothing to commit, working tree clean
```

But suppose, you haved staged the changed file and you want to discard the change, here restore must be used. Don't use checkout. It won't work here. See this:

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   this.txt

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git add .

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   this.txt


ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git checkout this.txt
Updated 0 paths from the index
```

here checkout won't work

So to do this:

```
git restore --staged fileName
git restore fileName
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
```

```
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   this.txt
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git restore --staged this.txt

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   this.txt

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git restore this.txt
```

Here note that: If you are using VS Code and unstaged a change and came back to last commit, then VS Code will show the unstaged changes of your files with unsaved tag. Don't save it if you want to get back to last commit and discard the new changes (it needs some practice to understand this clearly).

Git Ignore

If you want to ignore tracking a file, you have to use gitignore.

Make a file named "***.gitignore***" . This is a text file. The path added in this file will be avoided tracking by git.

If inside .gitignore I write

```
kawshik.txt
myenv/
*.jpg
kkp/paul/
kkp/kawshik.png
```

Here myenv folder, paul folder, kawshik.txt file, all .jpg files, kawshik.png file will be avoided tracking by git.

Now suppose if you tracked a file previously and want to stop tracking now, run

```
git rm -r --cached filenName
```

Otherwise, it won't stop tracking though you added to gitignore.

It's better to add gitignore and then create some file.

Deleting Git Repo

If you want to stop a directory from git, run `rm -rf .git`

This mainly delete the .git folder.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ rm -rf .git
```

Git Branch

Branching is very important to make experiments and then merge the experiments to the main. Here the main branch is called master. Unless we add and switch to a branch, our initial branch is always master branch. Whatever, lets create branches.

To create branch

```
git checkout -b branchName
```

After creating branch, git automatically switches to new branch. Here **branchName** I used is **issue1**.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch
* master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git checkout -b issue1
Switched to a new branch 'issue1'

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue1)
$ git branch
* issue1
  master
```

To view branches (* will be current branch)

```
git branch
```

To switch branch

```
git checkout switchToWhatBranch
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue1)
$ git branch
* issue1
  master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue1)
$ git checkout master
Switched to branch 'master'

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ |
```

To delete branch (here it is named develop)

If the branch named develop is already merged with the master, then run

```
git branch -d develop
```

If it it not merged with the master but you still want to delete, then run

```
git branch -d develop
git branch -D develop
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch
  develop
* master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch -d develop
error: The branch 'develop' is not fully merged.
If you are sure you want to delete it, run 'git branch -D develop'.

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch -D develop
Deleted branch develop (was 54179ee).
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch
* master
```

Merging and Merge Conflicts

Merge conflict means, suppose you are working in two branchs parallelly and made changes to same portion of a file or something, then git won't understand which changes to keep. This is called merge conflict. If merge conflict happens, git will make you choose about what changes should be kept.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch
  issue1
* master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git merge issue1
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$
```

```
index.html ×
index.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
   Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
6  <<<<<<< HEAD (Current Change)
7      <title>This is my initial title 3</title>
8  =======
9      <title>This is my issue1 title 3 - c7</title>
10 >>>>>>> issue1 (Incoming Change)
11 </head>
12 <body>
13
14 </body>
15 </html>
```

Here in VS Code you can choose "**Accept Current Change**"/ "**Accept Incoming Change**" etc or you can change it manually but editing code. I clicked on Accept Incoming Change:

```
index.html ●
```

```html
index.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>This is my issue1 title 3 - c7</title>
</head>
<body>

</body>
</html>
```

Now run git status, it will show I have resolved merge conflicts but didn't added the new changes.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git merge issue1
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$
```

We have to write

```
git add .
Or,
git add index.html
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
```

```
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$ git add .

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$ |
```

Then we have to commit. Then merging will be completed.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$ git add .

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   index.html


ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$ git commit -m "merged issue 1"
[master 12ee2ae] merged issue 1

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ |
```

We can see git log to see all commits including this. Run

```
git log
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master|MERGING)
$ git commit -m "merged issue 1"
[master 12ee2ae] merged issue 1

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git log
commit 12ee2ae1b614362784a8c61e75ae9ffc4a7370c2 (HEAD -> master)
Merge: 645441e 27035ce
Author: kawshikbuet17 <kawshik.kumar.paul@gmail.com>
Date:   Wed Nov 25 19:17:33 2020 +0600
```

```
    merged issue 1

commit 645441e08353a41c711dc98036523c9eab48e58e
Author: kawshikbuet17 <kawshik.kumar.paul@gmail.com>
```

We are inside git log now. To exit, type ‘q’.

To see branches and their last commits

```
git branch -v
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch
  issue1
  issue2
* master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch -v
  issue1 27035ce this is c7
  issue2 eb885aa made some problems
* master 12ee2ae merged issue 1
```

To see which branches are merged with me.

```
git branch --merged
git branch --no-merged
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch
  issue1
  issue2
* master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch -v
  issue1 27035ce this is c7
  issue2 eb885aa made some problems
* master 12ee2ae merged issue 1

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch --merged
  issue1
* master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch --no-merged
  issue2
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
```

Notice the difference of previous picture and this picture

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch --merged
  issue1
* master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch --no-merged
  issue2

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git checkout issue1
Switched to branch 'issue1'

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue1)
$ git branch --merged
* issue1

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue1)
$ git branch --no-merged
  issue2
  master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue1)
$ git checkout issue2
Switched to branch 'issue2'

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue2)
$ git branch --merged
  issue1
* issue2
  master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue2)
$ git branch --no-merged

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (issue2)
$ |
```

So, calculate the merge operation while in master, otherwise it will be difficult to calculate what is happening.

Here, issue1 is merged with the master, issue2 isn't. So deleting issue1 will make no loss but be alert while deleting issue2 because it is not merged. Before merging, deleting branch may occur some data losses. So be alert. Git will give error while deleting unmerged branch, won't give error while deleting already merged branch.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
```

```
$ git branch -d issue2
error: The branch 'issue2' is not fully merged.
If you are sure you want to delete it, run 'git branch -D issue2'.
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch -D issue2
Deleted branch issue2 (was eb885aa).
```

Pushing Git Branches To Remote Repositories

If your want to publish your repository, github is the most popular medium to do this. This is called git remote. Lets do this. If you are doing this for the first time, it is little stressful, but don't worry. Lets do this.

Configuring Github (Skip this step if you aren't first timer)

Firstly create a **github** account. Create Repository and get the link of that repository. Get the SSH link (not HTTPS). Here mine was

```
git@github.com:kawshikbuet17/GitTutorialDemo.git
```

Lets publish our git repo to github. Run the commands below.

```
git status
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
nothing to commit, working tree clean
```

working tree must be clean. If not, then commit your changes.

```
git remote add origin git@github.com:kawshikbuet17/GitTutorialDemo.git
```

```
git remote -v
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote -v
origin  git@github.com:kawshikbuet17/GitTutorialDemo.git (fetch)
```

```
origin  git@github.com:kawshikbuet17/GitTutorialDemo.git (push)
```

Okey. Added our remote. Now we can push it to our github (remote). Lets do it.

```
git push -u origin master
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git push -u origin master
The authenticity of host 'github.com (13.250.177.223)' can't be established.
RSA key fingerprint is SHA256:nThbg6kXUpJWGl7E1IGOCspRomTxdCARLviKw6E5SY8.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'github.com,13.250.177.223' (RSA) to the list of known hosts.
git@github.com: Permission denied (publickey).
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
```

It will not work for now. You have to tell your github about your PC that this is your PC. This is done by adding SSH key in github. SSH key options is in the settings of github. You have to run some commands into your git bash and get SSH key and then add it to your github account to verify your PC to your github account. After you do it once, you don't have to worry about this in future.

Run the following commands without thinking anything.

```
ssh-keygen -t rsa -b 4096 -C "kawshik.kumar.paul@gmail.com"
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ ssh-keygen -t rsa -b 4096 -C "kawshik.kumar.paul@gmail.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/c/Users/ASUS/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/ASUS/.ssh/id_rsa
Your public key has been saved in /c/Users/ASUS/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:sFuHcuLYjnwhV6grTGmgkMYFyVf48wMrZog3WpMyo8w kawshik.kumar.paul@gmail.com
The key's randomart image is:
+---[RSA [redacted]]----+
|[redacted]|
| o +            |
|..o . ..        |
|+o   +.o..      |
|[redacted]|
|[redacted] .    |
|[redacted]|
```

```
|[redacted]          |
|   .o..             |
+----[[redacted]]-----+
```

This output may not be appeared in your git bash. Someday I have done something maybe. but still follow the syntaxes.

```
eval $(ssh-agent -s)
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ eval $(ssh-agent -s)
Agent pid [redacted]
```

```
ssh-add ~/.ssh/id_rsa
```

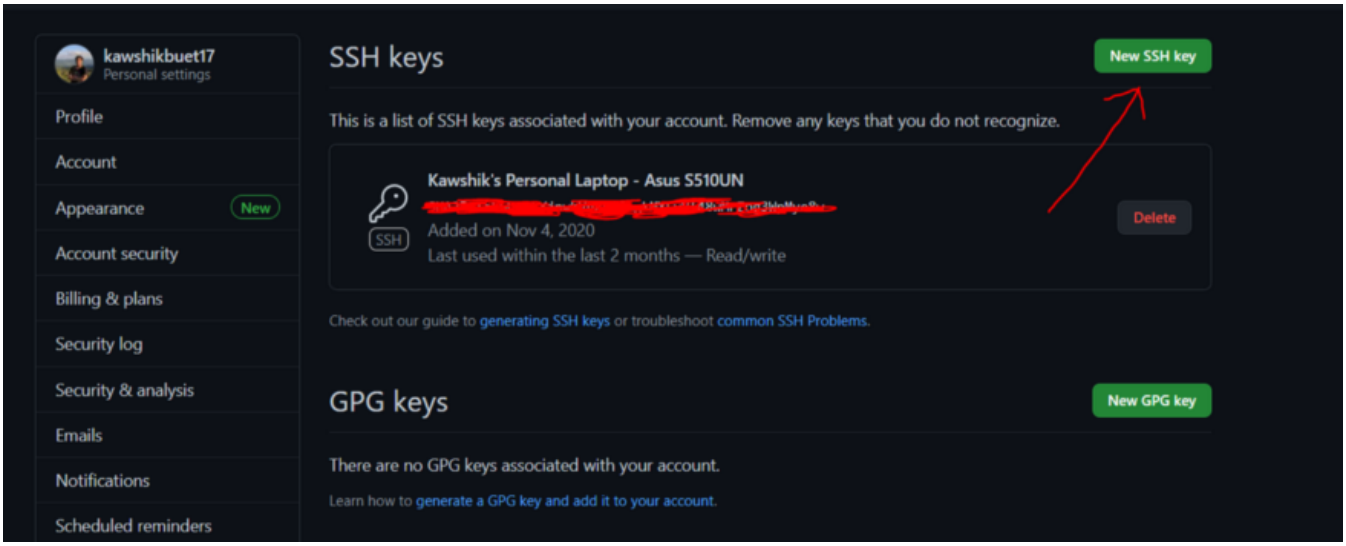
```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ ssh-add ~/.ssh/id_rsa
Identity added: /c/Users/ASUS/.ssh/id_rsa (kawshik.kumar.paul@gmail.com)
```

```
tail ~/.ssh/id_rsa.pub
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ tail ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAA[redacted]6oeDDq1teCI36M/b+
+a2Rn4gPjEzx3ozbjLvqsPEZ[redacted]bKgoPh/3FmxDoziOqXicreM
+kt3tEKblY4a3qxIAM7gPwWD[redacted]lA3REcep7ZmKhxYGUXqiQRxUZCN3cmBm2M1w9MCvu
+NVcfTKgfJRud2QzBi+1/6MY[redacted]iCrhqZGIzoKFA1I49Mu1Nl8wR1rPpDQX0OPsvF
+GI4cZ5Pu1FlvueMA7Eb54gI[redacted] kawshik.kumar.paul@gmail.com
```

Confidential Data

Copy this full key and add it to SSH key in github settings. Then you are done.



Now our push in github will work. Lets see.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git push -u origin master
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (11/11), 27.68 KiB | 2.77 MiB/s, done.
Total 11 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To github.com:kawshikbuet17/GitTutorialDemo.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

This is to be done for the first time. Later, you don't have to add SSH key. `git push -u origin master` will work directly.

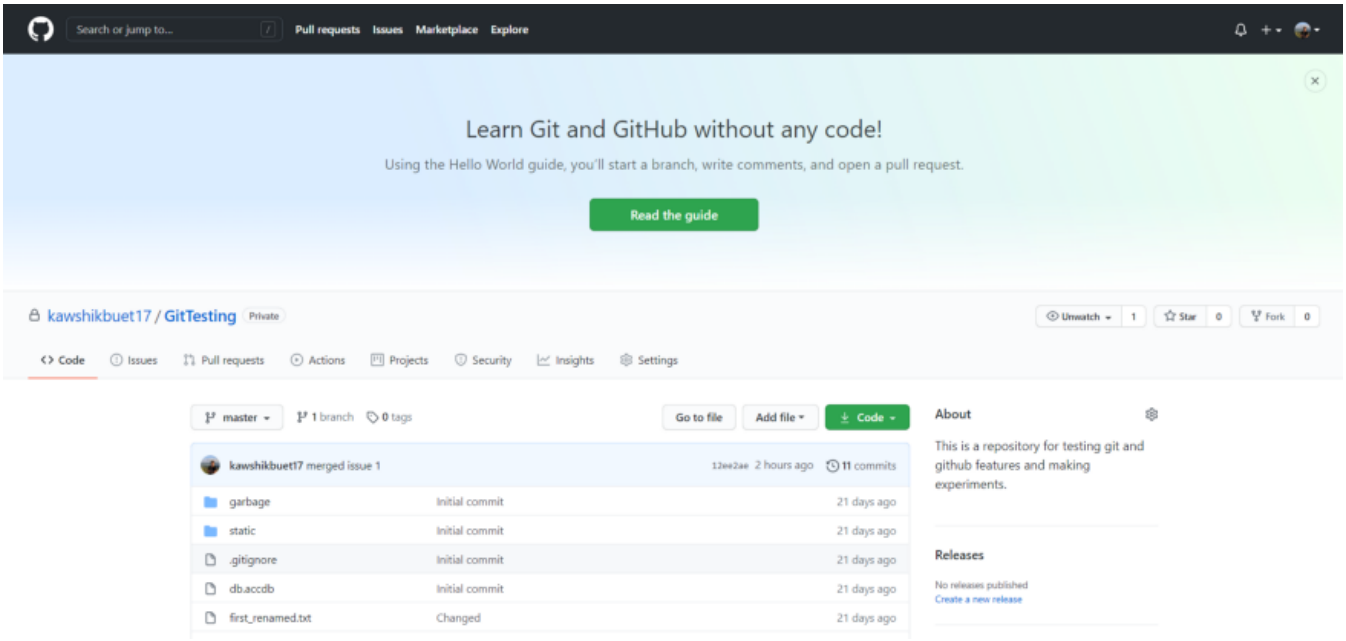
Working with Git Remote (GitHub)

If your have a git repository in your local machine, you can publish it to remote repository like this.

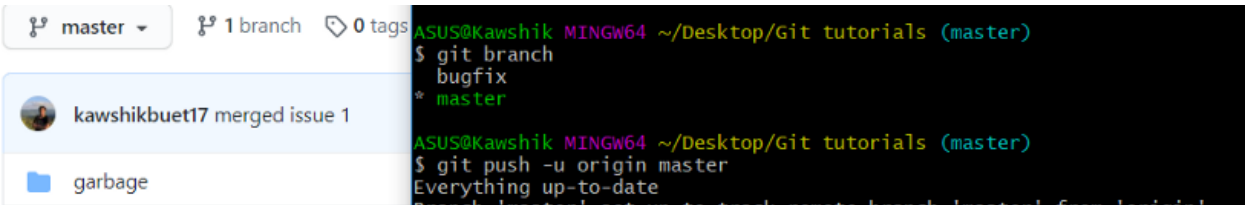
```
MINGW64:/c/Users/ASUS/Desktop/Git tutorials

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote -v

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote add origin https://github.com/kawshikbuet17/GitTesting.git

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote
origin

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote -v
origin  https://github.com/kawshikbuet17/GitTesting.git (fetch)
origin  https://github.com/kawshikbuet17/GitTesting.git (push)

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ |
```

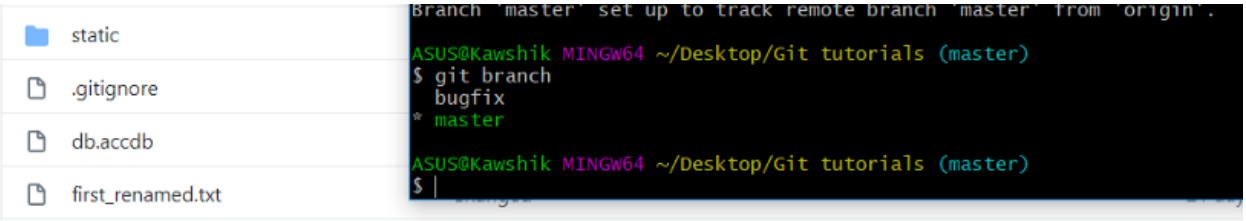
```
MINGW64:/c/Users/ASUS/Desktop/Git tutorials
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote
origin

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote -v
origin  https://github.com/kawshikbuet17/GitTesting.git (fetch)
origin  https://github.com/kawshikbuet17/GitTesting.git (push)

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git push -u origin master
Enumerating objects: 42, done.
Counting objects: 100% (42/42), done.
Delta compression using up to 8 threads
Compressing objects: 100% (37/37), done.
Writing objects: 100% (42/42), 30.15 KiB | 2.74 MiB/s, done.
Total 42 (delta 19), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (19/19), done.
To https://github.com/kawshikbuet17/GitTesting.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ |
```

Lets go to our github repository and see the repository. It will work fine if you don't make any mistake.



Here only master will be pushed. Other branches will not be pushed. We have to push other branches separately. Lets do it. Suppose we had a branch named bugfix.





First switch to the branch using

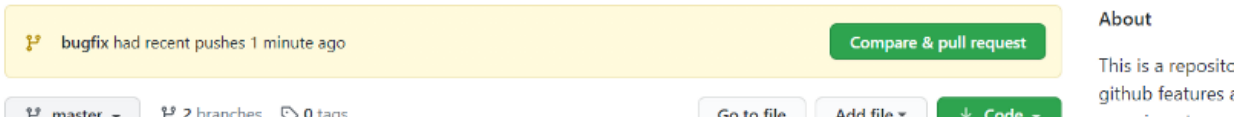
```
git checkout bugfix
```

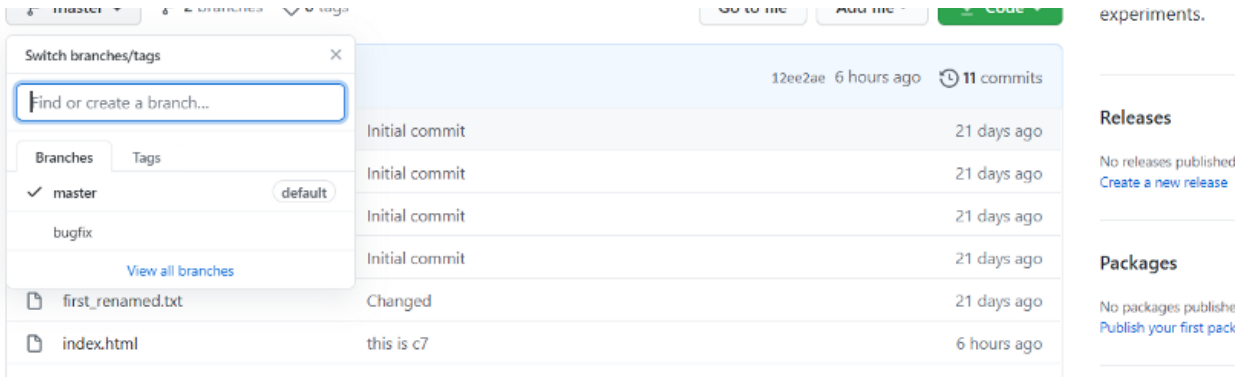
```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch
  bugfix
* master
```

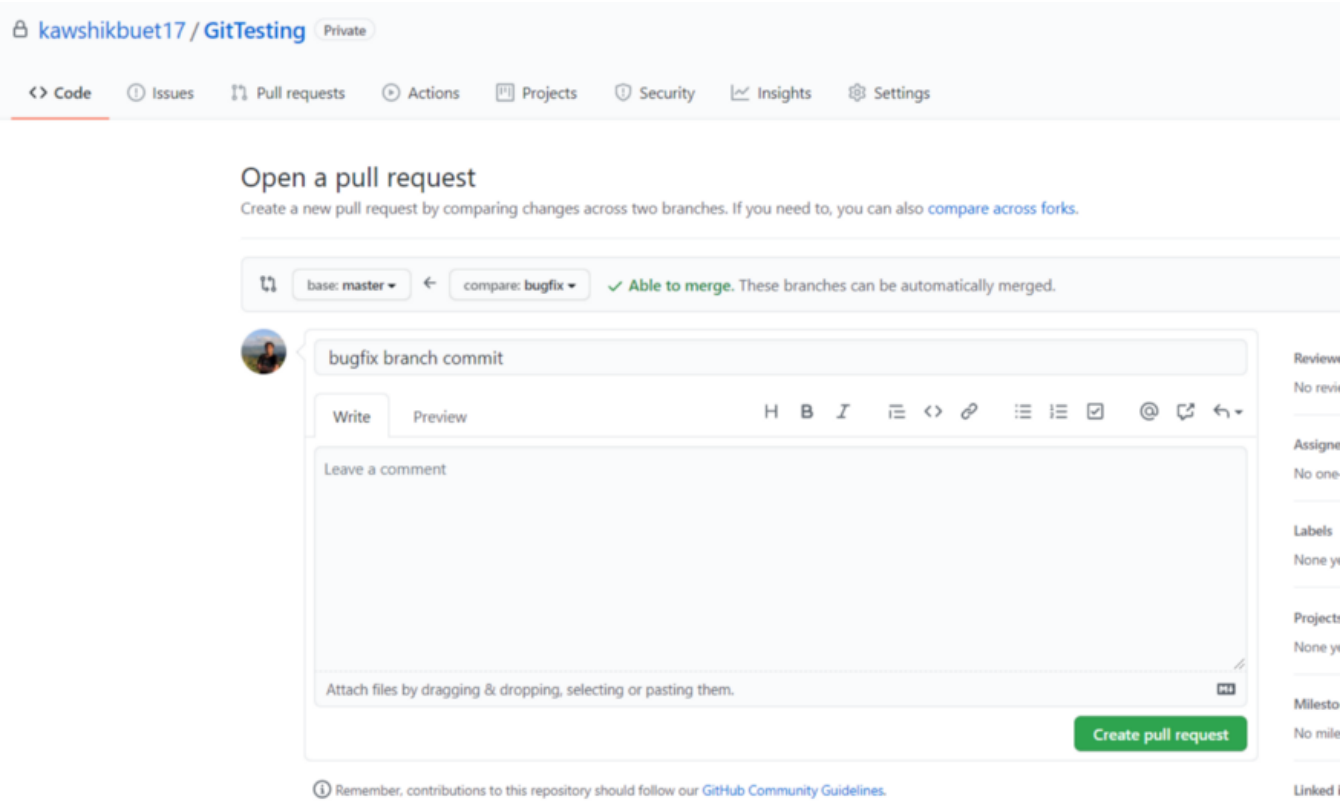
```
git push origin bugfix
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (bugfix)
$ git status
On branch bugfix
nothing to commit, working tree clean

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (bugfix)
$ git push origin bugfix
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 388 bytes | 388.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'bugfix' on GitHub by visiting:
remote:      https://github.com/kawshikbuet17/GitTesting/pull/new/bugfix
remote:
To https://github.com/kawshikbuet17/GitTesting.git
 * [new branch]      bugfix -> bugfix

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (bugfix)
$
```

Now here we can see that bugfix branch has been pushed to github.







Notice the Able to Merge sign

No commit comments for this range

If ‘able to merge’ arrives in this page, it means that we can merge it without any conflict.

Notice: While pushing a branch, be in that branch (It is recommended). Otherwise problems may occur. such as notice this thing:

```
om/kawshikbuet17/Git
 bugfix -> bugfix
```

```
m/kawshikbuet17/Gi
 master -> master
up to track remote
```

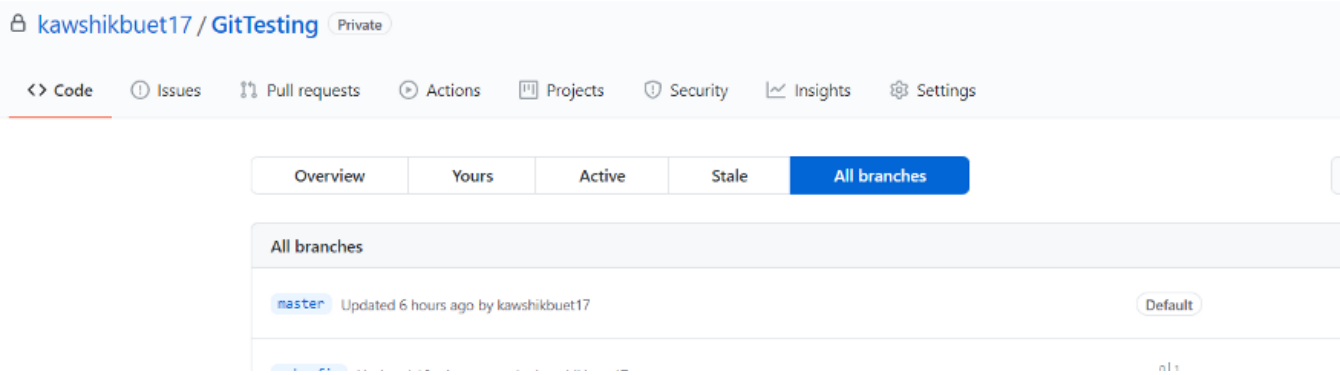
But if we do:

```
git push origin bugfix:mybugfix
```

Then our bugfix will be added into github with the name of mybugfix as another branch. This is dangerous.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (bugfix)
$ git push origin bugfix:mybugfix
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'mybugfix' on GitHub by visiting:
remote:      https://github.com/kawshikbuet17/GitTesting/pull/new/mybugfix
remote:
To https://github.com/kawshikbuet17/GitTesting.git
 * [new branch]      bugfix -> mybugfix
```

```
com/kawshikbuet17/GitTes
 bugfix -> mybugfix
```



mybugfix Updated 16 minutes ago by kawshikbuet17

bugfix Updated 16 minutes ago by kawshikbuet17

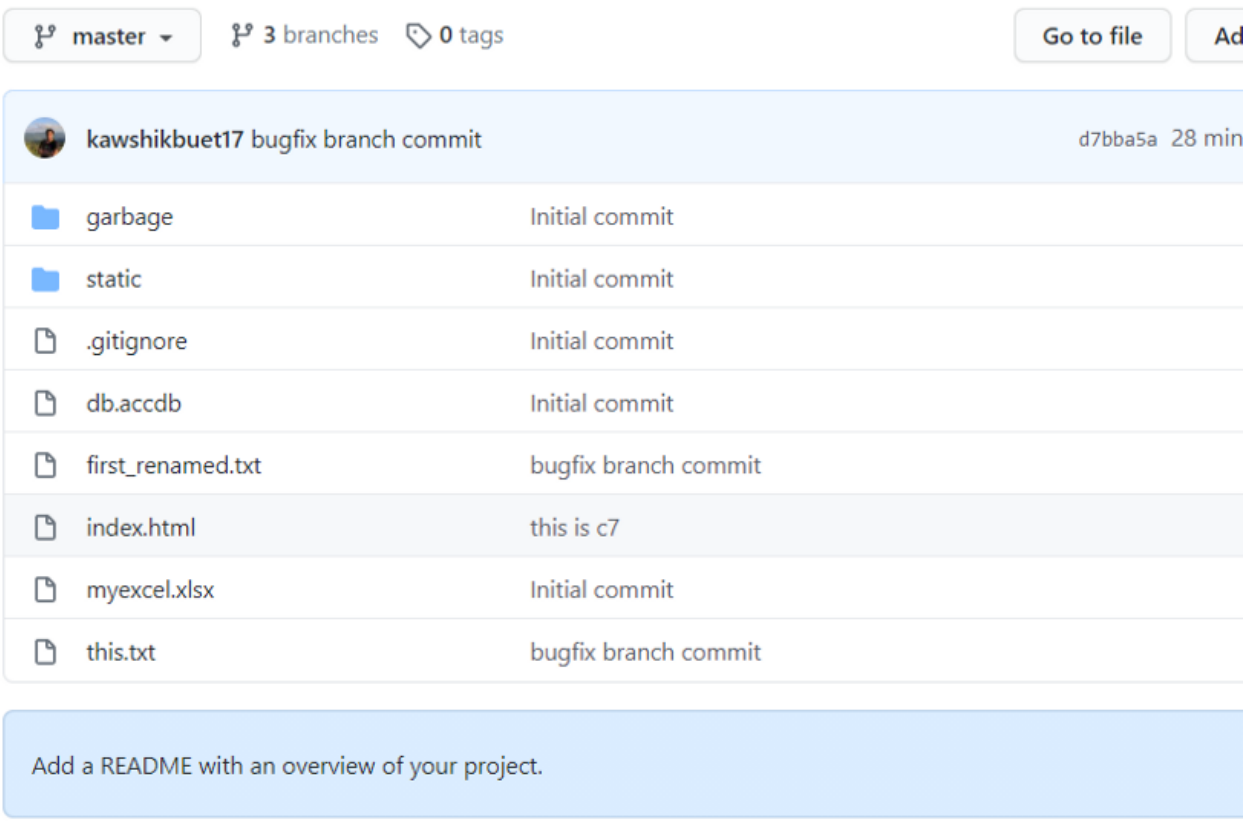
```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (bugfix)
$ git push origin bugfix:mybugfix
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'mybugfix' on GitHub by visiting:
remote:      https://github.com/kawshikbuet17/GitTesting/pull/new/mybugfix
remote:
To https://github.com/kawshikbuet17/GitTesting.git
 * [new branch]      bugfix -> mybugfix

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (bugfix)
$ git branch
* bugfix
  master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (bugfix)
$ |
```

But in local git, there is no mybugfix

Now we will merge bugfix and delete bugfix (if we delete before merge, we will get error). After merging, deleting bugfix and pushing, in github, still there will be bugfix. But merge is successful (I can see the commit of bugfix in master branch, notice this.txt)

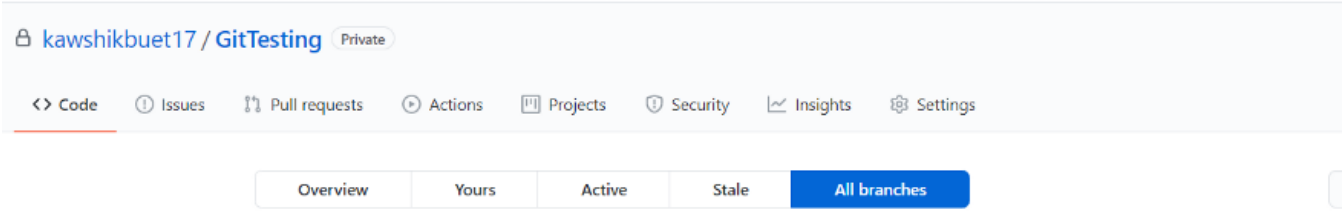
```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (bugfix)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch -d bugfix
error: The branch 'bugfix' is not fully merged.
If you are sure you want to delete it, run 'git branch -D bugfix'.

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git merge bugfix
Updating 12ee2ae..d7bba5a
Fast-forward
 first_renamed.txt | 3 +++
 this.txt          | 4 +++-
 2 files changed, 6 insertions(+), 1 deletion(-)

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

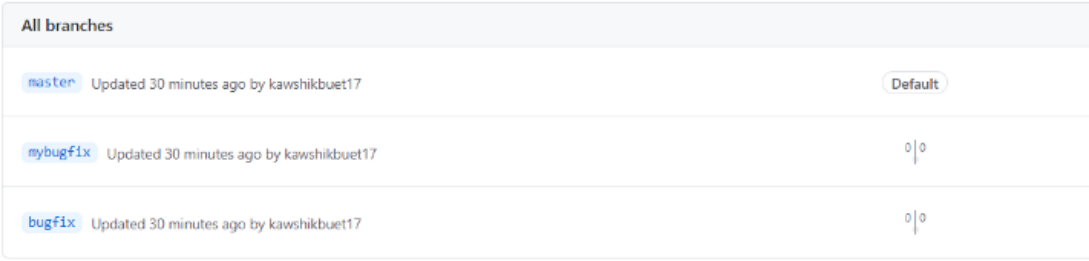
```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch -d bugfix
Deleted branch bugfix (was d7bba5a).

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git branch
* master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git push origin master
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/kawshikbuet17/GitTesting.git
   12ee2ae..d7bba5a  master -> master

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ |
```



bugfix are merged to master (notice this.txt commit)

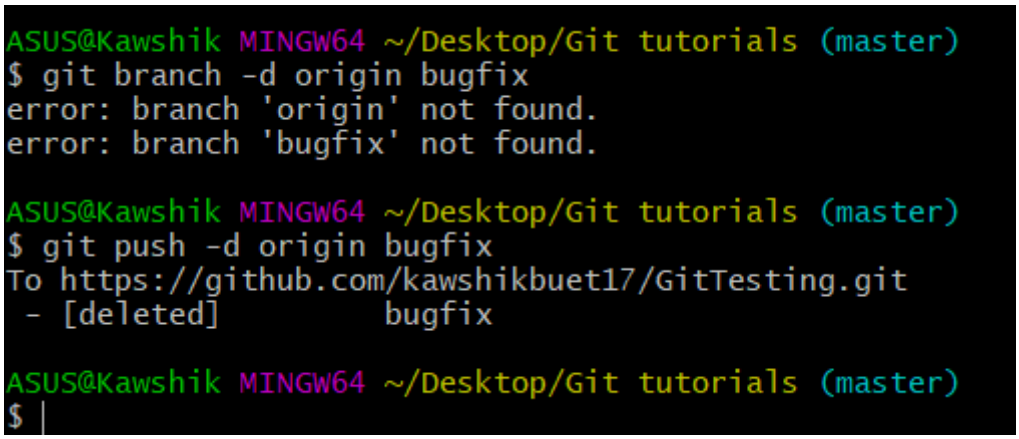
In github there are the branches but these are not in git (we deleted bugfix from git). But we didn't deleted from github, so it is showing still. Now we will delete it.

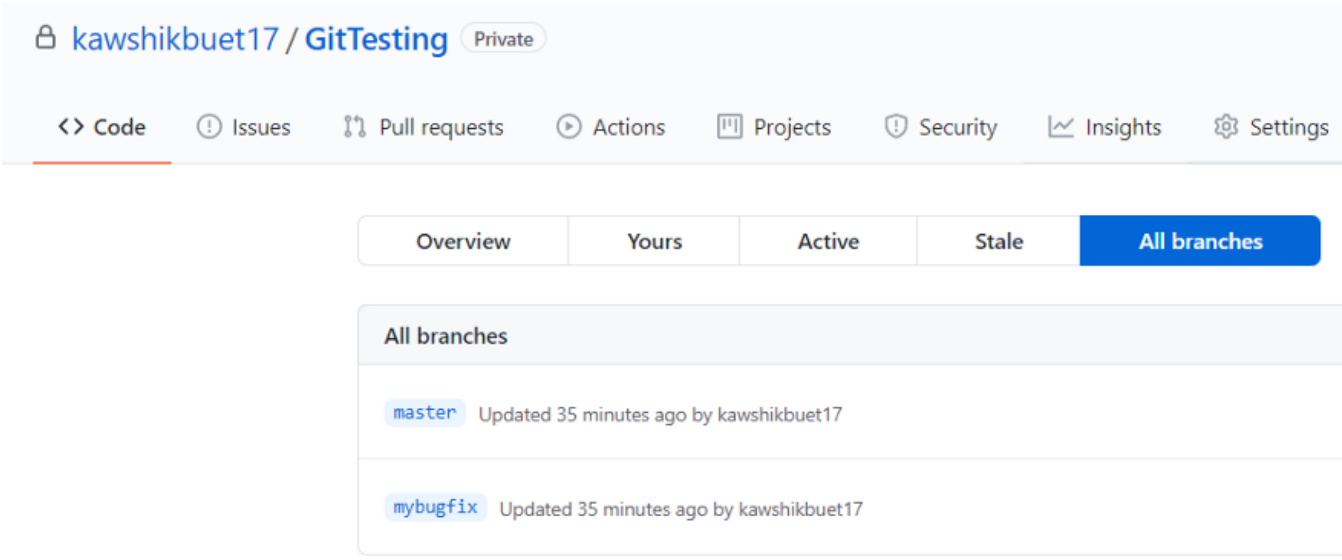


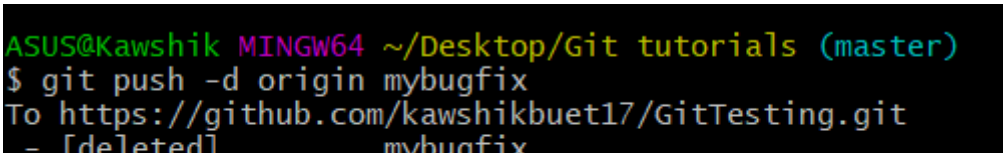


To delete branch from github (means git origin):

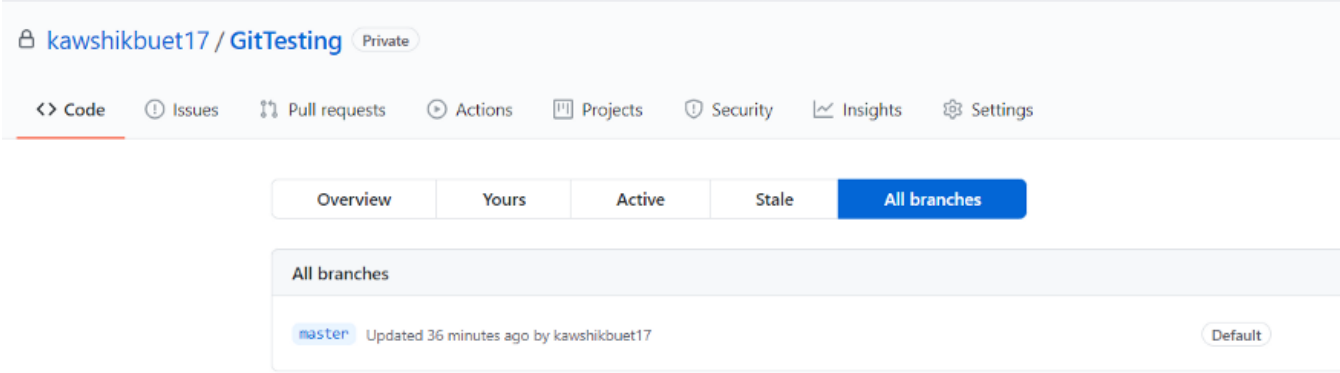
```
git push -d origin bugfix
```







```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ |
```



Delete Remote of Git

That means will will not access github repository for this git. to do this:

```
git remote rm origin
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote -v
origin  https://github.com/kawshikbuet17/GitTesting.git (fetch)
origin  https://github.com/kawshikbuet17/GitTesting.git (push)

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote rm origin

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote -v

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
```

It can be added again.

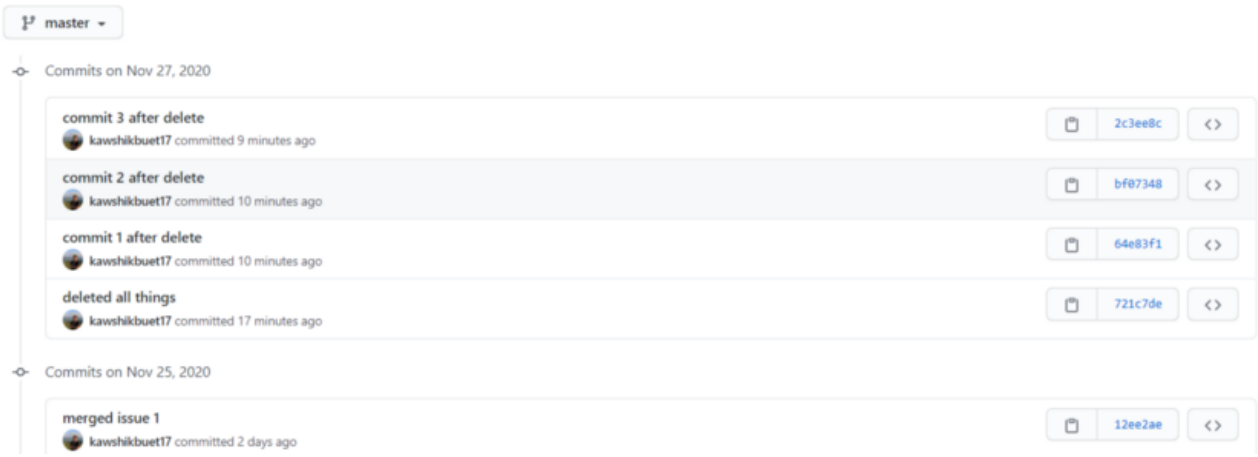
```
git remote add origin url_link_here
```

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote -v

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git remote add origin https://github.com/kawshikbuet17/GitTesting.git

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
```

```
$ git remote -v
origin  https://github.com/kawshikbuet17/GitTesting.git (fetch)
origin  https://github.com/kawshikbuet17/GitTesting.git (push)
```

Deleting Commits

Suppose we have to delete 4 commits from “commit 3 after delete” to “deleted all things” and go to the “merged issue 1” commit and discard the commits after it. How to do it?



Just get the commit id and run :

```
git reset --hard commit_id_here
```

Here my code will be:

```
git reset --hard 12ee2ae
```

After running this syntax you will get back to that commit. And you can work it from there. But if you published it, you must make a force push to make the changes in GitHub. To make that force push, run:

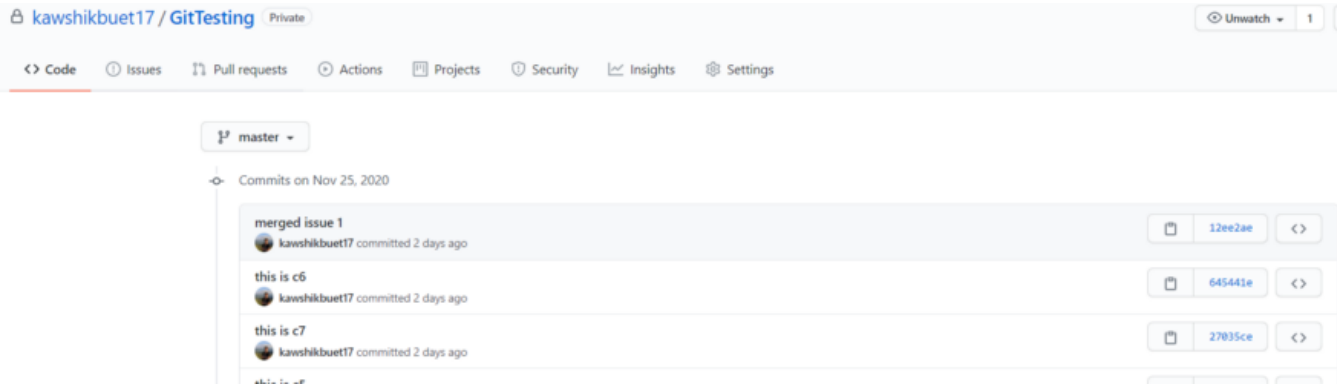
```
git push -f origin master
```

```
nothing added to commit but untracked files present (use "

ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git push -f origin master
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
```

```
To https://github.com/kawshikbuet17/GitTesting.git
 + 2c3ee8c...12ee2ae master -> master (forced update)
```

After force pushing, you will see that github will go to that point which you wanted. If you don't make force push, then changes will come to git, but will come in github. So you must make force push sometimes.

```
ASUS@Kawshik MINGW64 ~/Desktop/Git tutorials (master)
$ git log
commit 12ee2ae1b614362784a8c61e75ae9ffc4a7370c2 (HEAD -> master, origin/master)
Merge: 645441e 27035ce
Author: kawshikbuet17 <kawshik.kumar.paul@gmail.com>
Date:   Wed Nov 25 19:17:33 2020 +0600

    merged issue 1

commit 645441e08353a41c711dc98036523c9eab48e58e
Author: kawshikbuet17 <kawshik.kumar.paul@gmail.com>
Date:   Wed Nov 25 19:07:44 2020 +0600

    this is c6

commit 27035ce14a63c8dad35acc1b14321e363cf9ac76
```

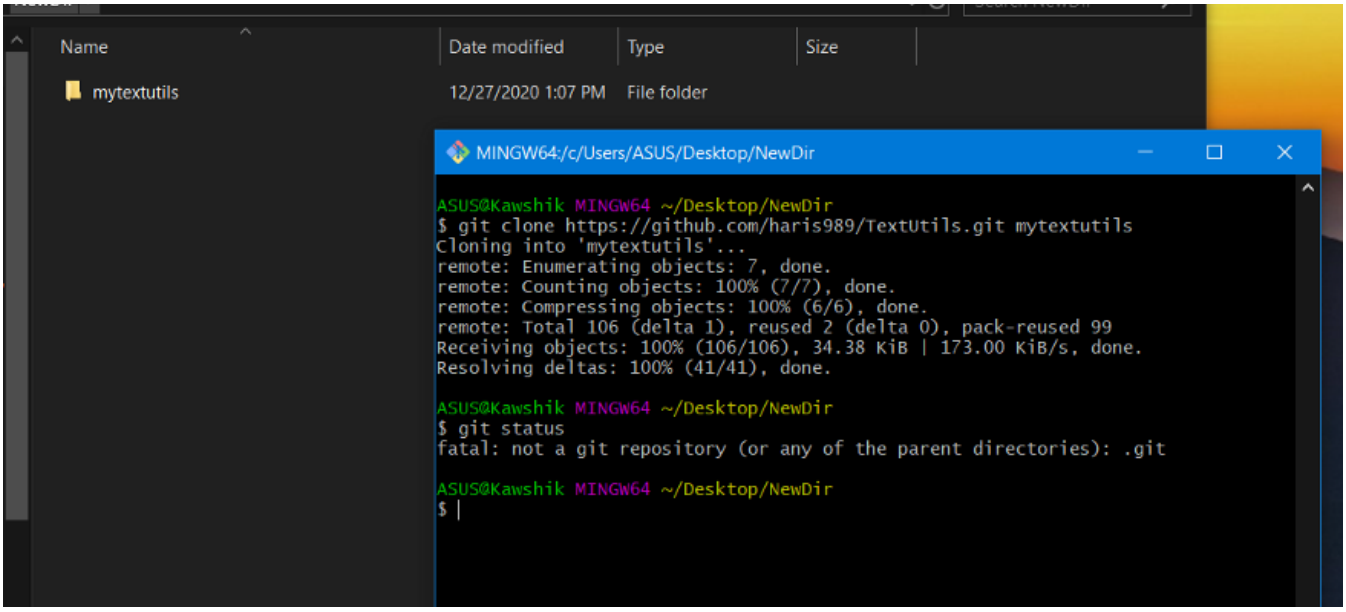


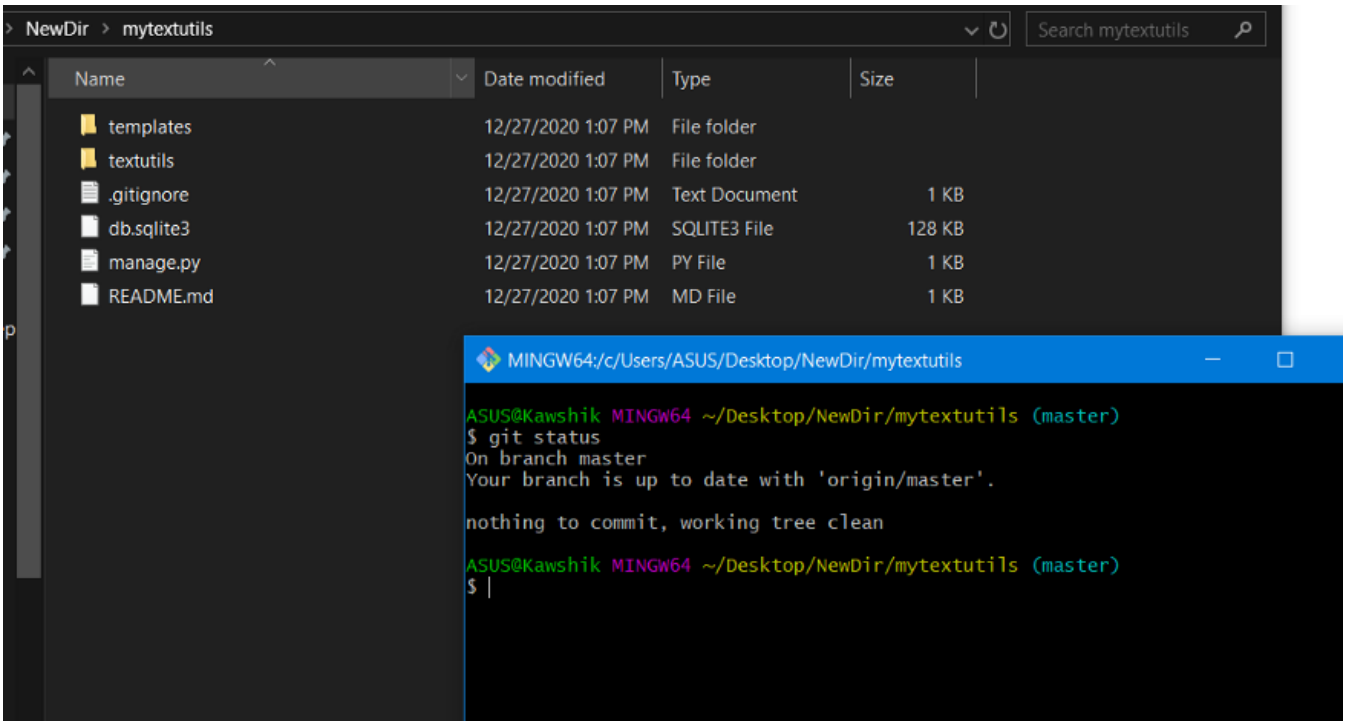
Git Clone

You can clone a public repository and work on it separately.

```
git clone https://github.com/haris989/TextUtils.git mytextutils
```

Here note that, while cloning in a directory (Here **NewDir**), the directory is not a git repo. But after cloning, get into the folder which is got by cloning. Then you will see that is a git repo.





You can do everything here which can be done in a git repo.

Btw, if you want to work in a project, one of your project partner must make a github repository and make you contributor. Then you have to clone that project and work on it. If you are a contributor, then you can make changes in master. You can make push and pull. Process are all same, but you need to know about pulling only. `git pull` to get the changes by others, `git push -u origin master` is to push your changes in github.
That's it.

I will develop this note time to time with many details and corrections. You can see more from this https://paste.ubuntu.com/p/vnyMsXbFTj/

Thanks for reading.

Get Kawshik Kumar Paul's stories in your inbox

You cannot subscribe to yourself

Git Github Remote Working Git Bash

About Help Legal

Get the Medium app

Download on the App Store

GET IT ON Google Play